

STRATEGI ALGORITMA
Tugas Besar Robocode
Suruh Ai Berpikir



Dipersiapkan oleh:

Dzaky Faris Al Faqih 124140077

Timothy Montoya Wilfried Mangapul Situngkir 124140104

Mohamad Rifky Putra 124140074

Program Studi Teknik Informatika

Institut Teknologi Sumatera

Tahun 2026

BAB 1 : Deskripsi tugas (dapat menyalin spesifikasi tugas ini).

Spesifikasi Wajib

- Buatlah 4 bot (1 utama dan 3 alternatif, **tetap dikumpulkan 1 bot yang terbaik saja**) dalam bahasa C# (**.net**) yang mengimplementasikan *algoritma Greedy* pada bot permainan Robocode Tank Royale dengan tujuan memenangkan permainan.
- Tugas dikerjakan berkelompok dengan anggota **minimal 2 orang** dan **maksimal 3 orang**.
- Strategi *greedy* yang diimplementasikan setiap kelompok harus dikaitkan dengan fungsi objektif dari permainan ini, yaitu memperoleh skor setinggi mungkin pada akhir pertempuran. Hal ini dapat dilakukan dengan mengoptimalkan komponen skor yang telah dijelaskan diatas.
- Strategi *greedy* yang diimplementasikan **harus berbeda** untuk setiap bot yang diimplementasikan dan setiap strategi *greedy* harus menggunakan **heuristic** yang berbeda.
- **Bot yang dibuat TIDAK BOLEH sama dengan SAMPEL yang diberikan sebagai CONTOH. Baik dari starter pack maupun dari repository engine asli.**
- Buatlah strategi *greedy* terbaik, karena setiap **bot utama** dari masing-masing kelompok akan diadu dalam kompetisi Tubes.
- Strategi *greedy* yang kelompok anda buat harus **dijelaskan dan ditulis secara eksplisit** pada laporan, karena akan diperiksa saat demo apakah strategi yang dituliskan sesuai dengan yang diimplementasikan.
- Setiap kelompok dapat menggunakan kreativitas yang bermacam macam dalam menyusun strategi *greedy* untuk memenangkan permainan. Implementasi pemain **harus dapat dijalankan pada *game engine*** yang telah disebutkan diatas serta dapat dikompetisikan dengan bot dari kelompok lain.
- Program harus mengandung komentar yang jelas, dan untuk setiap strategi *greedy* yang disebutkan, harus dilengkapi dengan **kode sumber yang dibuat**. Artinya semua strategi harus diimplementasikan
- Mahasiswa disarankan membaca dokumentasi dari *game engine*. Perlu diperhatikan bahwa game engine yang digunakan untuk tubes ini **SUDAH DIMODIFIKASI**. Untuk Perubahan dapat dilihat pada bagian starter pack.

Spesifikasi Bonus

- (maks 10) Membuat video tentang aplikasi *greedy* pada bot serta simulasinya pada game kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”, dll. **Semakin menarik video, maka semakin banyak poin yang diberikan.**
- (maks 10) Beberapa kelompok pemenang lomba kompetisi akan mendapatkan nilai tambahan berdasarkan posisi yang diraih.

- Jika terdapat kesulitan selama mengerjakan tugas besar sehingga memerlukan bimbingan, maka dapat melakukan asistensi tugas besar kepada asisten (opsional). Dengan catatan asistensi hanya bersifat membimbing, bukan memberikan “jawaban”.
- Bot yang telah dibuat akan dikompetisikan dengan kelompok lain dan disaksikan oleh seluruh peserta kuliah. Terdapat hadiah menarik bagi kelompok yang memenangkan kompetisi.
- Setiap kelompok harap mengisi nama kelompok dan anggotanya pada link berikut, paling lambat **Minggu, 26 April 2026**.

- RA : <https://forms.gle/vudRWyHFXzWZaiKA7>
- RB : <https://s.itera.id/TubesStima-RB26>
- RC : <https://s.itera.id/TubesStigma-RC26>
- RD : <https://forms.gle/z9Wa7rmfZJfxwQhY8>

- Program disimpan dalam repository yang bernama **Tubes_NamaKelompok** dengan nama kelompok sesuai dengan yang di sheets diatas. Berikut merupakan struktur dari isi repository tersebut:



- Folder src berisi semua source code
 - Folder doc berisi laporan tugas besar dengan format **NamaKelompok.pdf** dengan ketentuan isi pada **isi laporan**.
 - README untuk tata cara penggunaan yang minimal berisi:
 - Penjelasan singkat algoritma greedy yang diimplementasikan untuk setiap bot yang dibuat
 - Requirement program dan instalasi tertentu bila ada
 - Command atau langkah-langkah dalam meng-compile atau build program
 - Author (identitas pembuat)
- Sangat disarankan untuk menggunakan ***semantic commit***. Buatlah release dengan format **v1.x** dengan x adalah nomor revisi dimulai dari revisi 0. Contoh v1.0 untuk release pertama, v1.1 untuk revisi selanjutnya.
 - Pastikan untuk membuat repository bersifat **Public** paling lambat **H+1 deadline (1 hari setelah deadline)**. Sebelum deadline repository harus bersifat **Private**.
 - Laporan dikumpulkan hari **(menyusul)**.
PERINGATAN: Keterlambatan akan mengurangi nilai sebanyak 1 poin untuk setiap menit keterlambatan.
 - Adapun pertanyaan terkait tugas besar ini bisa disampaikan melalui QnA berikut:
[QnA Tubes STIMA 2026](#)

BAB 2 : Landasan Teori

2.1 Algoritma Greedy

Algoritma Greedy adalah suatu algoritma yang memecahkan persoalan secara perlangkah sedemikian sehingga, Seperti pada setiap langkah mengambil pilihan yang

terbaik yang dapat diperoleh saat itu tanpa memperhatikan sebuah konsekuensi ini mirip dengan prinsip “ambil yang kamu bisa ambil sekarang”, dan berharap bahwa dengan memilih optimum lokal pada setiap langkah akan berakhir dengan optimum global.

2.2 Cara Kerja Robocode Tank Royale API

Bagaimana cara kerja program secara umum bot pada mengimplementasikan algoritma greedy dalam bot :

Secara umum bot ini dibuat dari frameworknya Robocode Tank Royale API untuk .NET. program tersebut bekerja real time , artinya bot tidak hanya jalan lurus dari atas ke bawah melainkan merespons kejadian pada saat bertarung secara real time. Kita dapat Inisialisasi main dan run saat program dijalankan, fungsi main akan memicu bot untuk terhubung pada server robocode, sedangkan fungsi run itu yang kemudian mengatur warna visual bot dan juga mengaktifkan fitur rotasi independent agar radar , meriam , dan badan bot dapat berputar sendiri tanpa saling mengunci. Lalu bot juga dapat melakukan aksi dasar secara terus menerus dengan menggunakan loop while (isrunning), biasanya bot dapat memutar radar untuk mencari musuh di arena. Bot juga dapat merespon karena dapat mendeteksi sesuatu, server mengirimkan yang memicu bot untuk mengeluarkan fungsi khusus seperti OnScannedBot yang dipicu saat radar mendeteksi musuh nanti bot dapat memilih target, membidik, hingga dapat menentukan daya tembak, OnHitByBullet ini membuat bot jika tertembak dapat menghindari atau mengubah arahnya , OnHitWall dan OnHitBot ini dipicu agar saat menabrak bot atau dinding biar bot otomatis mundur agar tidak terjebak.

Untuk implementasinya Greedy yaitu adalah daripada menyerang secara acak bot kita menghitung sebuah Heuristic skor pada saat OnScannedBot terjadi , lalu bot dapat memilih skor tertinggi, bot kita menggunakan rumus pembagian. Nilai greedy akan naik jika energy musuh kecil dan jarak musuh dekat, kita juga menggunakan knapsack yang dimana bot menghitung value tinggi jika musuh mempunyai skor poin besar dan cost yang dimana rendah jika energi dihabiskan untuk menembak sedikit, bot juga dapat memilih target secara efisien energi terbaik saat itu juga, Greedy juga dapat menentukan daya tembak bot dan dalam bertahan hidup, bot dapat memakan energi bot sendiri oleh karena itu firepower antara 0.1 hingga 3.0 ditentukan secara greedy melalui CalculateFirePower yang dimana jika jarak musuh dekat bisa gunakan daya maksimal 3.0 karena peluang hampir 100% , jika musuh sudah mati bot menurunkan daya tembak. Lalu saat bertahan hidup jika bot darahnya dibatas aman maka bot mengabaikan target

ofensif dan fokus pada langkah instan untuk selamat seperti menjauh dari musuh terdekat atau beralih ke mode melingkar

BAB 3 :Aplikasi strategi greedy.

3.1 Proses Mapping Persoalan Robocode Tank Royale ke Elemen-Elemen Algoritma Greedy

Robocode Tank Royale adalah simulasi pertempuran tank berbasis aturan di mana setiap bot harus membuat keputusan secara real-time, memilih musuh yang akan diserang, mengatur daya tembak, menentukan pergerakan, dan menghindari tabrakan. Karena setiap giliran (turn) harus membuat keputusan segera tanpa kemampuan melihat ke depan secara eksak, persoalan ini sangat cocok dimodelkan dengan algoritma Greedy.

Algoritma Greedy bekerja dengan membuat keputusan terbaik secara lokal di setiap langkah tanpa mempertimbangkan konsekuensi jangka panjang secara menyeluruh. Dalam konteks Robocode Tank Royale, pemetaan elemen-elemen algoritma Greedy adalah sebagai berikut.

Elemen Greedy	Deskripsi	Implementasi Dalam Bot
Himpunan Kandidat	Semua musuh yang terdeteksi radar dan masih hidup dalam memori bot.	Dictionary<int, Enemy> enemies, menyimpan data semua bot musuh yang pernah di-scan dan belum mati/kedaluwarsa.
Himpunan Solusi	Satu musuh terpilih sebagai target utama per giliran (turn).	Variabel target yang dikembalikan oleh fungsi PickTarget() / KnapsackPickTarget() / SelectBestTarget().
Fungsi Seleksi	Memilih kandidat terbaik berdasarkan heuristik tertentu dari semua musuh yang ada.	Fungsi PickTarget(), KnapsackPickTarget(), SelectBestTarget(). Masing-masing menghitung skor dan memilih yang tertinggi.
Fungsi Kelayakan	Memastikan kandidat memenuhi syarat minimum agar layak dipilih.	Pengecekan: age <= ENEMY_MEMORY_TURNS, Energy > 0, distance <= threshold. Musuh yang terlalu jauh atau datanya kedaluwarsa diabaikan.
Fungsi Solusi	Menentukan apakah solusi (target terpilih) sudah cukup untuk bertindak.	Setelah target dipilih, bot langsung menjalankan tiga aksi: ControlRadar, AimAndShoot, ControlMovement. Solusi dianggap valid jika target tidak null.

Fungsi Objektif	Mengukur kualitas keseluruhan solusi. Tujuan akhir yang ingin dioptimalkan.	Memaksimalkan damage yang diberikan sambil meminimalkan damage yang diterima: kombinasi dari target dengan HP rendah, jarak dekat, dan posisi aman dari tembok.
-----------------	---	---

3.1.1 Himpunan kandidat

Himpunan kandidat adalah kumpulan semua musuh yang saat ini diketahui oleh bot melalui hasil pemindaian radar. Setiap kali radar mendeteksi musuh (event OnScannedBot), data musuh tersebut termasuk posisi X/Y, energi, kecepatan, arah gerak, dan waktu deteksi terakhir disimpan ke dalam struktur data Dictionary<int, Enemy>. Himpunan ini bersifat dinamis, musuh yang sudah mati atau data yang sudah kedaluwarsa (melebihi ENEMY_MEMORY_TURNS giliran) secara otomatis dihapus setiap awal giliran. Dengan demikian, himpunan kandidat selalu mencerminkan kondisi pertempuran yang paling mutakhir.

3.1.2 Himpunan Solusi

Himpunan solusi dalam konteks ini adalah satu musuh terpilih yang menjadi target utama pada giliran tersebut. Keputusan pemilihan target bersifat serakah (greedy). Bot memilih satu kandidat terbaik dari himpunan kandidat berdasarkan fungsi seleksi, tanpa mempertimbangkan apakah pilihan tersebut optimal dalam jangka panjang. Target yang terpilih kemudian digunakan sebagai referensi untuk tiga aksi utama yaitu pengarahan radar, penembakan, dan penentuan pergerakan.

3.1.3 Fungsi Seleksi

Fungsi seleksi adalah mekanisme inti algoritma Greedy yang menentukan kandidat mana yang paling layak dipilih pada iterasi saat ini. Fungsi ini mengevaluasi setiap musuh dalam himpunan kandidat dan mengembalikan yang memiliki nilai heuristik tertinggi. Keempat alternatif solusi yang dikembangkan masing-masing memiliki fungsi seleksi yang berbeda, mencerminkan pendekatan heuristik yang beragam.

3.1.4 Fungsi Kelayakan (Feasibility Function)

Tidak semua kandidat dalam himpunan kandidat layak untuk dipilih. Fungsi kelayakan menyaring kandidat yang tidak memenuhi syarat minimum. Syarat kelayakan yang diterapkan meliputi umur data musuh tidak melebihi batas memori ($age \leq ENEMY_MEMORY_TURNS$), energi musuh masih lebih dari nol ($Energy > 0$), dan (3) jarak musuh tidak melebihi jangkauan efektif penembakan ($distance \leq threshold$ tertentu, misalnya 700-1200 unit). Kandidat yang tidak lolos saringan ini akan dilewati meskipun ada dalam himpunan kandidat.

3.1.5 Fungsi Solusi

Fungsi solusi menentukan apakah solusi yang telah terpilih sudah cukup untuk dieksekusi. Dalam implementasi bot ini, solusi dianggap valid dan siap dieksekusi apabila fungsi seleksi berhasil mengembalikan target yang tidak null. Jika tidak ada target valid (semua kandidat tidak lolos uji kelayakan), bot akan beralih ke perilaku default misalnya memutar radar secara penuh dan berpatroli hingga target baru ditemukan.

3.1.6 Fungsi Objektif

Fungsi objektif merepresentasikan tujuan akhir yang ingin dioptimalkan oleh bot secara keseluruhan. Memenangkan pertandingan dengan memaksimalkan damage yang diberikan kepada musuh sekaligus meminimalkan damage yang diterima. Secara lebih spesifik, fungsi objektif diimplementasikan secara implisit melalui kombinasi tiga strategi yaitu memilih target yang paling menguntungkan untuk ditembak, menggunakan daya tembak yang optimal sesuai situasi, dan mempertahankan posisi yang aman dari serangan balik dan tabrakan tembok.

3.2 Eksplorasi 4 Alternatif Solusi Greedy

3.2.1 Alternatif 1: Nearest Enemy + Chase & Ram Heuristic (RamFireJr / Bot 1)

Deskripsi Heuristik

Alternatif pertama menggabungkan dua heuristik sederhana. Pilih musuh yang jaraknya paling dekat, lalu kejar dan tabrak (Chase & Ram). Intuisi di balik pendekatan ini adalah bahwa musuh terdekat paling realistis untuk dikejar, dan damage kontak (ramming) adalah cara paling pasti untuk memberikan damage tanpa memerlukan akurasi tembak. Bot hanya menembak saat sudah benar-benar dekat ($\text{jarak} \leq 120$ unit), sehingga hit chance mendekati 100%. Alternatif 1 memiliki konsep yang mirip dengan sample bot RamFire, yaitu mengejar dan menabrak musuh. Namun Alternatif 1 dikembangkan dengan strategi greedy yang lebih dinamis, yaitu memilih musuh terdekat sebagai target utama berdasarkan data musuh yang tersimpan. Selain itu, bot ini melakukan scanning, penguncian radar, penguncian gun, dan penembakan secara lebih terkontrol dibandingkan RamFire yang hanya bereaksi langsung terhadap musuh yang sedang terdeteksi.

Fungsi seleksi Alt-1:

```
target = argmin { DistanceTo(e.X, e.Y) | e ∈ enemies, e.Energy > 0 }
```

Setelah target dipilih, bot menjalankan tiga fungsi utama secara berurutan setiap giliran: LockRadar (mengunci radar ke target), LockGunAndFire (mengarahkan senjata dan menembak hanya jika jarak ≤ 120 dan gun sudah sejajar), dan ChaseAndRam (memutar badan ke arah target dan maju kecepatan maksimal 8). Saat bertabrakan dengan bot lain, kecepatan dipertahankan (terus dorong). Saat menabrak tembok, bot mundur sejenak ($\text{TargetSpeed} = -8$).

Kelebihan: Strategi sangat agresif dan efektif membunuh musuh di jarak dekat. Damage ramming tidak memerlukan amunisi, sehingga hemat energi senjata. Implementasi sederhana dengan logika yang mudah dipahami. Kekurangan: Mengabaikan HP musuh, bot akan tetap mengejar musuh berenergi penuh sambil mengabaikan musuh yang hampir mati. Tidak ada evasion movement; bot menjadi target tembak yang mudah saat mengejar musuh dari jarak jauh.

3.2.2 Alternatif 2: Multi-Criteria Greedy Score (SetOrbit / Bot 2)

Deskripsi Heuristik

Alternatif kedua menggunakan heuristik skor multi-kriteria yang menimbang lima faktor sekaligus yaitu kecepatan musuh (rendah = lebih mudah ditembak), HP musuh (rendah = lebih dekat ke eliminasi), jarak dari jarak ideal, umur data, dan bonus situasional (musuh hampir mati, energi rendah). Formula skor adalah:

$$\begin{aligned} \text{score} = & 2.2 \times (1 - \min(1, |\text{speed}|/8)) \times 0.45 \\ & + 1.8 \times (\max(0, 100 - \text{HP}) / 100) \\ & + 1.5 \times (1 - \min(1, |\text{dist} - \text{idealDist}| / 500)) \\ & + 1.3 \times (1 - \text{age} / \text{MEMORY_TURNS}) \\ & + \text{bonus_situasional} - \text{penalty_bahaya} - \text{age} \times 0.22 \end{aligned}$$

Setelah target dipilih, daya tembak ditentukan secara adaptif (fungsi GetFirePower), dan pergerakan dievaluasi dengan kandidat arah orbit yang memaksimalkan skor gabungan jarak-tembak dan jarak-ideal dari musuh. Fungsi AimAndShoot memperkirakan posisi musuh saat peluru tiba (linear prediction) sebelum menembak. Bot juga mendukung manuver ramming oportunistik jika musuh hampir mati dan jarak sangat dekat.

Kelebihan: Seimbang dan tidak mudah dieksploitasi. Mempertimbangkan posisi, HP, ancaman, dan keamanan secara bersamaan. Pergerakan orbit membuat bot lebih sulit ditembak. Kekurangan: Bobot heuristik membutuhkan tuning manual. Kompleksitas lebih tinggi dibanding Alt-1 dan Alt-4 meskipun tetap $O(n)$.

3.2.3 Alternatif 3: Knapsack Efficiency Ratio (Knapsack / Bot 3)

Deskripsi Heuristik

Alternatif ketiga mengadaptasi prinsip Knapsack (0/1 Knapsack Problem) ke dalam konteks pemilihan target. Setiap musuh memiliki "nilai" (value) yang merepresentasikan seberapa berharga membunuh musuh tersebut, dan "biaya" (cost) yang merepresentasikan berapa banyak energi yang harus dihabiskan. Heuristik memilih musuh dengan rasio Value/Cost tertinggi, analog dengan memilih item berbobot rendah namun bernilai tinggi dalam knapsack.

$$\begin{aligned} \text{Value} &= \text{killValue} \times \text{hitChance} + \text{bonus}(\text{nearlyDead}) + \text{bonus}(\text{closeRange}) \\ \text{Cost} &= \text{bulletsNeeded} \times \text{powerPerShot} \\ \text{Ratio} &= \text{Value} / \max(1, \text{Cost}) \leftarrow \text{pilih yang Ratio tertinggi} \end{aligned}$$

Daya tembak juga dipilih secara knapsack. Jika musuh hampir mati, gunakan daya tembak minimum yang cukup untuk membunuhnya (hemat energi untuk target berikutnya). Pergerakan bersifat dua mode. Agresif (mengejar dan menyerang musuh ber-HP rendah) dan aman (orbit menjaga jarak saat energi sendiri rendah).

Kelebihan: Pengelolaan energi paling efisien; tidak membuang amunisi untuk target yang mahal. Strategi mode aman membuat bot bertahan lebih lama di akhir pertandingan. Kekurangan: Dalam situasi banyak musuh ber-HP tinggi, bot mungkin terlalu pasif karena semua target dianggap 'mahal'. Kalkulasi cost bisa menjadi kurang akurat jika musuh bermanuver tidak terduga.

3.2.4 Alternatif 4 — Energy+Score Weighted Heuristic (GreedyBot / Bot 4)

Deskripsi Heuristik

Alternatif keempat menggunakan weighted sum sederhana dari dua atribut musuh. Energi musuh (HP) dan skor musuh. Premis heuristik ini adalah bahwa musuh berenergi tinggi dan berskor tinggi merupakan ancaman terbesar sekaligus memberikan reward paling besar jika berhasil dieliminasi.

$$\text{HeuristicScore} = (\text{EnemyEnergy} \times \text{WEIGHT_ENERGY}) + (\text{EnemyScore} \times \text{WEIGHT_SCORE})$$

→ default: WEIGHT_ENERGY = 1.0, WEIGHT_SCORE = 0.5

Bot ini memiliki mode agresif otomatis ketika energi sendiri turun dibawah AGGRO_THRESHOLD (40): selalu menembak dengan daya maksimum, radar berputar lebih cepat, dan bot menerobos musuh saat bertabrakan. Tidak ada pergerakan strategis khusus, bot bergantung sepenuhnya pada agresivitas penembakan.

Kelebihan: Heuristik sangat transparan dan mudah disesuaikan (hanya ubah bobot). Mode agresif di akhir pertandingan bisa efektif mengejutkan musuh. Kekurangan: Memilih musuh ber-HP tinggi justru berarti memilih musuh yang paling susah dibunuh. Tidak ada evasion movement bot rentan terkena banyak tembakan.

3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi Greedy

Aspek	Alt-1: Nearest Enemy + Chase & Ram (TemplateBot / Bot 1)	Alt-2: Multi-Criteria Score (SetOrbit / Bot 2)	Alt-3: Knapsack Ratio (KnapsackBot / Bot 3)	Alt-4: Energy+Score Heuristic (GreedyBot / Bot 4)
Heuristik	Pilih musuh terdekat (jarak minimum), lalu	Skor gabungan: kecepatan, HP, jarak ke ideal,	Rasio Value/Cost: nilai kill dibagi	Skor = (EnemyEnergy × WEIGHT_ENER

Aspek	Alt-1: Nearest Enemy + Chase & Ram (TemplateBot / Bot 1)	Alt-2: Multi-Criteria Score (SetOrbit / Bot 2)	Alt-3: Knapsack Ratio (KnapsackBot / Bot 3)	Alt-4: Energy+Score Heuristic (GreedyBot / Bot 4)
	kejar dan tabrak (Chase & Ram).	umur data, bonus one-shot.	energi yang dihabiskan.	GY) + (EnemyScore $\times$ WEIGHT_SCORE). Musuh ber-HP tinggi dan skor tinggi diprioritaskan karena berpotensi memberikan damage lebih besar jika ditembak.
Daya Tembak	Hanya tembak saat jarak ≤ 120 dan gun sudah sejajar; daya tembak maks (3.0) untuk hit chance tertinggi.	Adaptif: disesuaikan HP musuh, jarak, dan energi diri sendiri.	Adaptif: minimal power jika musuh hampir mati (hemat energi).	Normal: interpolasi linear antara MIN_FIRE_POWER (1.0) dan MAX_FIRE_POWER (3.0) berdasarkan jarak. Aggressive mode (energy < 40): selalu tembak MAX_FIRE_POWER.
Pergerakan	Chase & Ram: kejar musuh kecepatan maks (8), terus dorong saat bertabrakan, mundur saat kena tembok.	Orbit optimal: evaluasi kandidat sudut gerakan vs tembok dan jarak.	Agresif vs aman tergantung energi diri; chase jika musuh HP rendah.	Tidak memiliki pergerakan strategis khusus. Saat menabrak tembok, mundur lalu berbelok. Saat aggressive mode, menabrak musuh

Aspek	Alt-1: Nearest Enemy + Chase & Ram (TemplateBot / Bot 1)	Alt-2: Multi-Criteria Score (SetOrbit / Bot 2)	Alt-3: Knapsack Ratio (KnapsackBot / Bot 3)	Alt-4: Energy+Score Heuristic (GreedyBot / Bot 4)
				dan langsung tembak.
Kelebihan	Sangat agresif dan efektif membunuh musuh jarak dekat; damage kontak tidak perlu amunisi.	Menyeimbangkan banyak faktor, tidak mudah dieksploitasi.	Hemat energi: tidak buang amunisi untuk target yang mahal.	Heuristik sangat simpel dan transparan. Mode agresif berguna di akhir pertandingan.
Kekurangan	Mengabaikan HP musuh dan bahaya posisi; rentan terkena banyak tembakan saat mengejar musuh kuat.	Bobot heuristik perlu tuning, kompleksitas kalkulasi lebih tinggi.	Dalam skenario agresif banyak musuh, nilai kill bisa terlalu konservatif.	Memilih musuh ber-HP tinggi justru lebih sulit dibunuh, tidak mempertimbangkan jarak atau bahaya posisi. Tidak ada evasion movement.

3.3.1 Analisis Kompleksitas Komputasi

Semua keempat alternatif memiliki kompleksitas $O(n)$ dalam hal pemilihan target, di mana n adalah jumlah musuh yang diketahui. Perbedaannya terletak pada konstanta kalkulasi: Alt-1 hanya menghitung satu nilai (jarak), Alt-4 menghitung dua nilai (energy, score), sementara Alt-2 dan Alt-3 menghitung formula multi-variabel. Namun karena n (jumlah musuh) dalam Robocode Tank Royale umumnya kecil (kurang dari 10), perbedaan kompleksitas ini tidak berdampak signifikan pada performa real-time.

3.3.2 Analisis Efisiensi Energi

Efisiensi energi mengacu pada seberapa hemat bot menggunakan amunisi untuk menghasilkan eliminasi. Alt-3 (Knapsack) unggul dalam dimensi ini karena secara eksplisit memodelkan trade-off energi yang dihabiskan versus nilai yang didapat. Bot akan menembak dengan daya minimum yang cukup untuk membunuh musuh yang hampir mati, dan menyimpan

energi untuk target berikutnya. Alt-2 (Multi-Criteria) juga cukup efisien karena daya tembak adaptif berdasarkan jarak dan HP. Alt-1 dan Alt-4 lebih boros karena tidak mempertimbangkan efisiensi energi secara eksplisit.

3.3.3 Analisis Kemampuan Bertahan (Survival)

Kemampuan bertahan dipengaruhi oleh kualitas pergerakan evasion. Alt-2 memiliki sistem pergerakan paling canggih: evaluasi kandidat arah orbit dengan mempertimbangkan jarak dari tembok dan sudut lateral terhadap musuh, sehingga bot selalu bergerak membuat sudut yang menyulitkan penembakan musuh. Alt-3 juga memiliki mode safe orbit yang aktif saat energi rendah. Alt-1 hampir tidak memiliki evasion movement, bot hanya mengejar musuh lurus-lurus, menjadikannya target tembak yang mudah. Alt-4 juga rentan karena tidak memiliki gerakan menghindar.

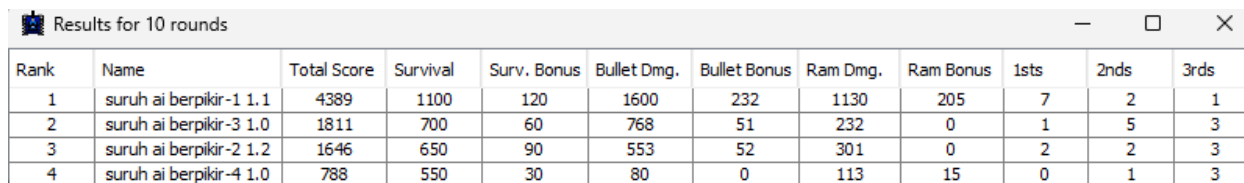
3.3.4 Analisis Efektivitas dalam Pertandingan Multi-Lawan

Dalam pertandingan dengan banyak lawan (melee), kemampuan memprioritaskan target yang benar menjadi kritis. Alt-2 dan Alt-3 unggul karena keduanya mempertimbangkan HP musuh: target yang hampir mati mendapat prioritas lebih tinggi, memaksimalkan peluang eliminasi. Alt-1 mudah terganggu oleh musuh kuat yang kebetulan dekat, strategi chase & ram efektif dalam 1v1, namun di melee bot akan terus mengejar satu musuh sambil terkena tembakan dari musuh lain. Alt-4 justru memilih musuh ber-HP tinggi yang paling sulit dieliminasi, membuat energi terbuang sia-sia.

3.4 Strategi Greedy yang Dipilih dan Alasan Pemilihannya

Berdasarkan hasil pengujian yang dilakukan pada keempat bot dengan format 1v1v1v1, strategi yang dipilih untuk diimplementasikan sebagai strategi utama adalah Alternatif 1.

Pemilihan Alternatif 1 didasarkan pada hasil percobaan selama 10 ronde, di mana bot Alternatif 1 memperoleh skor tertinggi dibandingkan alternatif lainnya. Bot alternatif 1 mendapatkan total skor sebesar 4389 dengan kemenangan ronde terbanyak yaitu 7 kali peringkat pertama. Hasil ini menunjukkan bahwa strategi agresif dengan memilih musuh terdekat, mengejar, menabrak, dan menembak pada jarak dekat memberikan performa paling efektif dalam skenario pengujian yang dilakukan. Berikut adalah hasil percobaan yang dilakukan



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	suruh ai berpikir-1 1.1	4389	1100	120	1600	232	1130	205	7	2	1
2	suruh ai berpikir-3 1.0	1811	700	60	768	51	232	0	1	5	3
3	suruh ai berpikir-2 1.2	1646	650	90	553	52	301	0	2	2	3
4	suruh ai berpikir-4 1.0	788	550	30	80	0	113	15	0	1	3

Gambar 3.1 Hasil Percobaan Bot

3.4.1 Alasan Pemilihan

Pemilihan Alternatif 1 sebagai strategi utama didasarkan pada pertimbangan berikut:

- Alternatif 1 memperoleh skor tertinggi dari bot yang diuji. Selisih skor dengan alternatif lain cukup besar, sehingga dapat disimpulkan bahwa strategi ini paling efektif dalam kondisi pertandingan yang digunakan. Selain total skor bot tertinggi, bot ini juga berhasil meraih posisi sebanyak 7 kali dari 10 ronde.
- Strategi Alternatif 1 berfokus pada musuh terdekat sebagai target utama. Dengan mendekati target secara agresif, peluang tembakan menjadi lebih tinggi karena jarak tembak semakin pendek. Selain itu, strategi ramming memberikan damage tambahan tanpa bergantung sepenuhnya pada akurasi tembakan. Hal ini membuat bot mampu memberikan tekanan besar kepada musuh secara langsung.
- Meskipun heuristik yang digunakan relatif sederhana, yaitu memilih musuh berdasarkan jarak terdekat, hasil menunjukkan pendekatan ini sangat efektif. Strategi yang sederhana ini membuat bot dapat mengambil keputusan dengan cepat tanpa perhitungan heuristik yang kompleks.
- Dengan terus mengejar dan menekan musuh terdekat, bot dapat memaksa lawan masuk ke situasi yang tidak menguntungkan. Musuh yang dikejar akan lebih sulit menjaga jarak, mengatur posisi, dan melakukan serangan balik secara optimal. Strategi ini membuat Alternatif 1 menghasilkan kombinasi damage dari tembakan jarak dekat dan tabrakan.

3.4.2 Pertimbangan terhadap Alternatif Lain

Alternatif 2 memiliki keunggulan dari sisi keseimbangan strategi karena menggunakan multi-criteria score dan pergerakan orbit. Namun, berdasarkan hasil pengujian, bot hanya memperoleh skor 1646, jauh di bawah Alternatif 1. Hal ini menunjukkan bahwa strategi orbit yang lebih aman belum mampu menandingi tekanan agresif dari chase and ram.

Alternatif 3 memperoleh skor sekitar 1811 dan menjadi salah satu pesaing terkuat setelah Alternatif 1. Strategi berbasis efisiensi energi cukup baik karena mampu mempertimbangkan value dan cost dalam memilih target. Namun, pendekatan ini cenderung lebih berhati-hati, sehingga dalam pertandingan agresif bot tidak selalu mampu menghasilkan eliminasi secepat Alternatif 1.

Alternatif 4 memperoleh skor terendah, yaitu sekitar 788. Strategi ini kurang efektif karena pemilihan target berdasarkan energi dan skor musuh dapat menyebabkan bot menyerang musuh yang justru sulit dikalahkan. Selain itu, tidak adanya evasion movement yang kuat membuat bot lebih mudah terkena serangan lawan.

BAB 4 : Implementasi dan pengujian.

4.1 Alternatif 1: Nearest Enemy + Chase & Ram Heuristic (RamFireJr / Bot 1)

ALGORITMA RamFireJr

DEKLARASI

enemies : daftar musuh
target_terbaik : musuh
jarak_minimum : real
jarak_musuh : real

DESKRIPSI

SELAMA pertandingan masih berjalan LAKUKAN

Bersihkan musuh yang sudah lama tidak terlihat atau sudah mati

target_terbaik $\leftarrow$ NULL
jarak_minimum $\leftarrow \infty$

UNTUK setiap musuh DALAM enemies LAKUKAN

jarak_musuh $\leftarrow$ hitungJarak(bot, musuh)

JIKA jarak_musuh < jarak_minimum MAKA

jarak_minimum $\leftarrow$ jarak_musuh
target_terbaik $\leftarrow$ musuh

ENDIF

ENDFOR

JIKA target_terbaik = NULL MAKA

putarRadar()
jalankanBotDenganKecepatanRendah()
janganMenembak()

ELSE

arahkanRadarKe(target_terbaik)
arahkanGunKe(target_terbaik)

JIKA jarak_minimum $\leq$ 120 DAN gunSudahSejajar() DAN senjataSiap() MAKA

tembakDenganFirepowerMaksimum()
ENDIF

arahkanBadanKe(target_terbaik)
majuDenganKecepatanMaksimum()

ENDIF

ENDWHILE

Bot menggunakan struktur data Dictionary bernama enemies untuk menyimpan data musuh yang terdeteksi. Untuk setiap musuh dalam objek EnemyInfo yang berisi BotId, Posisi x dan y,

Energy, Direction, speed dan LastSeen. Fungsi utama greedy-nya adalah PickNearestTarget(), yaitu memilih musuh dengan jarak paling dekat dari bot, Sebelum memilih target, bot menjalankan CleanEnemies() untuk menghapus data musuh yang sudah mati ataupun sudah lama tidak terlihat. Setelah target ditemukan, bot menjalankan prosedur LockRadar() untuk mengarahkan radar ke target, LockGunAndFire() untuk mengarahkan senjata dan menembak jika target dekat, serta ChaseAndRam() untuk mengejar dan menabrak target. Singkatnya, bot ini selalu memilih musuh terdekat sebagai target utama, lalu langsung mengejar, mengunji, dan menyerang.

4.2 Alternatif 2: Multi-Criteria Greedy Score (SetOrbit / Bot 2)

ALGORITMA SetOrbit

DEKLARASI

enemies : daftar musuh
target_terbaik : musuh
skor_terbaik : real
skor_musuh : real
kandidat_gerak : daftar arah dan kecepatan
gerak_terbaik : arah dan kecepatan

DESKRIPSI

SELAMA pertandingan masih berjalan LAKUKAN

Hapus data musuh yang sudah lama tidak terlihat atau sudah mati

target_terbaik $\leftarrow$ NULL

skor_terbaik $\leftarrow -\infty$

UNTUK setiap musuh DALAM enemies LAKUKAN

JIKA data musuh masih valid MAKA

Hitung jarak musuh dari bot

Hitung skor musuh berdasarkan:

- kecepatan musuh
- energi musuh
- kedekatan dengan jarak ideal
- seberapa baru musuh terlihat
- peluang untuk menghabisi musuh

JIKA skor_musuh > skor_terbaik MAKA

skor_terbaik $\leftarrow$ skor_musuh

target_terbaik $\leftarrow$ musuh

ENDIF

ENDIF

ENDFOR

JIKA target_terbaik = NULL MAKA

Patrol atau putar radar untuk mencari musuh

ELSE

```
Arahkan radar ke target_terbaik
Arahkan gun ke prediksi posisi target_terbaik
```

```
JIKA gun sejajar, senjata siap, dan jarak memungkinkan MAKA
    Tembak dengan firepower sesuai jarak dan energi
ENDIF
```

```
Buat beberapa kandidat arah gerak:
- mengorbit kanan
- mengorbit kiri
- mundur menjauh
- maju mendekat
```

```
gerak_terbaik ← kandidat pertama
skor_terbaik ←  $-\infty$ 
```

```
UNTUK setiap kandidat gerak LAKUKAN
    Prediksi posisi bot jika bergerak ke arah tersebut
    Hitung skor gerak berdasarkan:
    - jarak dari dinding
    - jarak ideal dari musuh
    - kemampuan mengorbit target
    - risiko terlalu dekat atau terlalu jauh
```

```
    JIKA skor_gerak > skor_terbaik MAKA
        skor_terbaik ← skor_gerak
        gerak_terbaik ← kandidat gerak
    ENDIF
ENDFOR
```

```
    Gerakkan bot sesuai gerak_terbaik
ENDIF
```

```
ENDWHILE
```

Bot menggunakan struktur data Dictionary bernama enemies untuk menyimpan data musuh yang terdeteksi. Untuk setiap musuh dalam objek EnemyInfo yang berisi BotId, Posisi x dan y, Energy, Direction, speed dan LastSeen. Fungsi greedy utama adalah PickTarget(). Fungsi ini memilih musuh dengan skor tertinggi yang dihitung dari beberapa faktor seperti energi musuh, kecepatan musuh, jarak dari bot, kesesuaian dengan jarak ideal dan seberapa baru musuh terlihat. Greedy kedua pada ControlMovement(), fungsi ini memilih arah gerak terbaik dari beberapa kandidat arah mulai dari orbit kanan, orbit kiri, menjauh, atau mendekat. Setiap kandidat diberi skor berdasarkan keamanan dari dinding, jarak ideal terhadap musuh, dan kemampuan menjaga posisi orbit. Prosedur pendukung ControlRadar() untuk mengunci radar ke target, AimAndShoot() untuk mengarahkan senjata dan menembak berdasarkan prediksi posisi musuh, serta GetFirePower() untuk menentukan kekuatan tembakan sesuai jarak dan energi. Jika tidak ada target, bot menjalankan Patrol() untuk bergerak sambil mencari musuh. Singkatnya bot ini memilih target dengan skor terbaik, lalu memilih arah gerak dengan skor terbaik agar bot tetap aman sambil menjaga jarak orbit ideal dari musuh.

4.3 Alternatif 3: Knapsack Efficiency Ratio (Knapsack / Bot 3)

ALGORITMA KnapsackBot

DEKLARASI

SAFE_ENERGY : konstanta = 25
CLOSE_RANGE : konstanta = 220
enemies : daftar musuh (menyimpan koordinat X/Y, energi, kecepatan, arah, dan waktu terakhir terlihat)
target_terbaik : musuh
rasio_terbaik : real
arah_gerak : integer (bernilai 1 atau -1 untuk arah putar balik)
waktu_putar_balik : integer

DESKRIPSI

SELAMA pertandingan masih berjalan LAKUKAN

// PEMBERSIHAN MEMORI

Hapus data musuh yang sudah kadaluwarsa (> 12 turn tidak terlihat) atau mati ($\text{energi} \leq 0$)

target_terbaik $\leftarrow$ NULL

rasio_terbaik $\leftarrow -\infty$

// FASE 1: PEMILIHAN TARGET BERDASARKAN METODE KNAPSACK (VALUE / COST)

UNTUK setiap musuh DALAM enemies LAKUKAN

JIKA musuh valid DAN jarak musuh ke bot ≤ 1200 MAKA

// 1. Hitung Keuntungan (Value) - Seberapa berharga musuh ini dihabisi

peluang_kena $\leftarrow 1.0 - (\text{jarak_musuh} / 800)$

nilai_bunuh $\leftarrow$ Maksimal dari (0, $100 - \text{energi_musuh}$)

value $\leftarrow (\text{nilai_bunuh} * \text{peluang_kena})$

+ (Bonus jika energi musuh < 20)

+ (Bonus ekstra jika energi musuh < 8)

+ (Bonus jika musuh berada di CLOSE_RANGE)

// 2. Hitung Biaya (Cost) - Estimasi pengeluaran energi bot kita

estimasi_peluru $\leftarrow$ Maksimal dari (1, $\text{energi_musuh} / 4$)

power_per_tembakan $\leftarrow$ (Tentukan berdasarkan jarak: makin dekat makin besar)

cost $\leftarrow$ estimasi_peluru * power_per_tembakan

// 3. Evaluasi Rasio Efisiensi (Value per Cost)

rasio $\leftarrow$ value / cost

// 4. Mode Bertahan (Safe Mode Penalty)

JIKA energi bot $< \text{SAFE_ENERGY}$ MAKA

rasio $\leftarrow$ rasio * (10.0 / cost) // Target mahal jadi sangat tidak *worth*

// Simpan target dengan rasio keuntungan tertinggi

JIKA rasio $>$ rasio_terbaik MAKA

rasio_terbaik $\leftarrow$ rasio

target_terbaik $\leftarrow$ musuh

ENDIF

ENDIF

ENDFOR

```

// FASE 2: EKSEKUSI AKSI BERDASARKAN TARGET
JIKA target_terbaik = NULL MAKA
    Lakukan Patroli (hindari dinding, berjalan menuju area tengah)
    Putar radar secara maksimal (45 derajat) mencari musuh

ELSE
    // A. Logika Radar
    Arahkan dan kunci putaran radar secara presisi mengikuti koordinat target_terbaik

    // B. Logika Penembakan & Manajemen Energi
    Hitung jarak bot ke target_terbaik
    Tentukan kekuatan tembakan (pwr):
        JIKA energi bot < SAFE_ENERGY MAKA
            pwr ← rendah (0.5 hingga 0.8) demi menghemat energi
        ATAU JIKA energi target_terbaik < 4 MAKA
            pwr ← secukupnya saja untuk membunuh (jangan overspend)
        LAINNYA
            pwr ← sesuaikan dengan jarak (maksimal 3.0 untuk jarak dekat)

    Prediksi koordinat masa depan target_terbaik menggunakan rumus *Linear Targeting* (memperhitungkan waktu tempuh
    peluru, arah, dan kecepatan target)
    Arahkan laras ke koordinat prediksi tersebut

    JIKA laras sudah sejajar, laras tidak panas, dan sisa energi memadai MAKA
        Tembakkan peluru dengan kekuatan pwr
    ENDIF

    // C. Logika Pergerakan (Agro vs Safe Mode)
    JIKA energi bot < SAFE_ENERGY MAKA
        // Mode Kritis: Menjauh dari musuh
        Panggil SafeOrbit (mengorbit melebar/menjauh dari target_terbaik, putar balik tiap 25 turn)

    ELSE
        // Mode Sehat: Agresif / Menghindar
        JIKA (waktu sekarang - waktu_putar_balik) > 30 MAKA
            arah_gerak ← arah_gerak * -1 // Putar balik arah agar tembakan musuh meleset
            waktu_putar_balik ← waktu sekarang
        ENDIF

        Buat kandidat arah pergerakan:
            JIKA energi target_terbaik < 15 MAKA
                kandidat ← Arah agresif (maju mengejar target_terbaik)
            LAINNYA
                kandidat ← Arah strafing (menyamping 90 derajat atau 110 derajat dari target)

        // Evaluasi Kandidat Pergerakan
        gerak_terbaik ← kandidat pertama
        skor_gerak_terbaik ← -∞

    UNTUK setiap kandidat gerak LAKUKAN
        Simulasikan posisi bot jika maju ke arah tersebut
        Hitung skor gerak:
            - Beri penalti negatif ekstrim jika menabrak/terlalu dekat dinding
            - Beri bonus jika posisinya berada di jarak ideal (sekitar 250 unit dari musuh)

```

```

        JIKA skor gerak > skor_gerak_terbaik MAKA
            skor_gerak_terbaik ← skor gerak
            gerak_terbaik ← kandidat
        ENDIF
    ENDFOR

    Putar badan ke arah gerak_terbaik
    Gerakkan bot dengan kecepatan dari gerak_terbaik dikalikan arah_gerak
ENDIF

ENDIF

```

ENDWHILE

Pada implementasi KnapsackBot, struktur data *Class* Enemy dan Dictionary yang digunakan untuk manajemen memori musuh sama persis dengan yang ada pada bot sebelumnya. Perbedaan mendasar terletak pada fungsi penyeleksian targetnya, yaitu KnapsackPickTarget(). Fungsi ini tidak hanya melihat energi terbesar, melainkan mengadaptasi prinsip *Fractional Knapsack* dengan menghitung rasio efisiensi antara *Value* (keuntungan membunuh, diukur dari kerentanan HP musuh dan jarak) berbanding dengan *Cost* (estimasi pengeluaran energi bot untuk menembak). Fungsi ini secara *greedy* akan memilih target dengan rasio keuntungan tertinggi. Setelah fungsi mengembalikan target terbaik, prosedur Shoot() akan dijalankan untuk mengkalkulasi arah tembakan prediktif beserta manajemen dayanya, di mana prosedur ini secara cerdas menahan tembakan besar agar tidak membuang energi (*overspend*) pada musuh yang sudah sekarat. Untuk taktik bertahan hidup, prosedur pergerakannya didesain jauh lebih dinamis. Bot menggunakan prosedur Move() untuk bergerak agresif mengejar atau menyampingi musuh saat kondisi energi maksimal. Namun, jika energi bot jatuh ke batas kritis, bot akan memanggil prosedur SafeOrbit() yang mengubah pola geraknya menjadi orbit menjauh untuk menghindari peluru lawan secara maksimal. Semua siklus evaluasi, pembersihan memori, dan eksekusi tindakan ini dikoordinasikan secara berkelanjutan di dalam prosedur Tick().

4.4 Alternatif 4: Energy+Score Weighted Heuristic (GreedyBot / Bot 4)

ALGORITMA GreedyBot

DEKLARASI

```

enemies : daftar musuh (menyimpan koordinat, energi, kecepatan, arah, dan waktu terakhir terlihat)
target_terbaik : musuh
skor_terbaik : real
skor_musuh : real
kandidat_gerak : daftar arah dan kecepatan
gerak_terbaik : arah dan kecepatan
arah_gerak : integer (menyimpan nilai 1 atau -1 untuk manuver putar balik)
waktu_putar_balik : integer

```

DESKRIPSI

SELAMA pertandingan masih berjalan LAKUKAN

Hapus data musuh yang sudah lama tidak terlihat (> 12 turn) atau energinya <= 0

```
target_terbaik ← NULL
skor_terbaik ← -∞
```

```
// FASE 1: PEMILIHAN TARGET BERDASARKAN GREEDY
```

```
UNTUK setiap musuh DALAM enemies LAKUKAN
```

```
  JIKA data musuh masih valid MAKA
```

```
    Hitung skor_musuh berdasarkan:
```

```
      -  $(\text{Energi musuh} \times \text{Bobot Energi}) + (\text{Energi musuh} \times \text{Bobot Skor})$ 
```

```
    // Strategi greedy: incar yang nilainya paling menguntungkan (lokal optimum)
```

```
    JIKA skor_musuh > skor_terbaik MAKA
```

```
      skor_terbaik ← skor_musuh
```

```
      target_terbaik ← musuh
```

```
    ENDIF
```

```
  ENDIF
```

```
ENDFOR
```

```
// FASE 2: PENGAMBILAN KEPUTUSAN (RADAR, TEMBAKAN, DAN PERGERAKAN)
```

```
JIKA target_terbaik = NULL MAKA
```

```
  Lakukan Patroli (bergerak perlahan menjauhi dinding menuju tengah arena)
```

```
  Putar radar secara penuh (45 derajat) untuk mencari musuh
```

```
ELSE
```

```
  // Logika Radar
```

```
  Arahkan dan kunci putaran radar agar selalu mengikuti koordinat target_terbaik
```

```
  // Logika Penembakan (Predictive Shooting)
```

```
  Hitung jarak bot ke target_terbaik
```

```
  Tentukan kekuatan tembakan berdasarkan jarak dan status energi bot (agresif jika energi menipis)
```

```
  Prediksi posisi target_terbaik di masa depan berdasarkan:
```

```
    - Jarak dan kecepatan peluru
```

```
    - Kecepatan dan arah gerak target_terbaik saat ini
```

```
  Arahkan laras (gun) ke koordinat prediksi target
```

```
  JIKA laras sudah sejajar sesuai toleransi kemiringan, senjata tidak panas, dan energi bot cukup MAKA
```

```
    Tembak dengan kekuatan tembakan yang sudah ditentukan
```

```
  ENDIF
```

```
  // Logika Pergerakan (Strafing & Wall Avoidance)
```

```
  JIKA (waktu sekarang - waktu_putar_balik) > 30 MAKA
```

```
    arah_gerak ← arah_gerak * -1 // Putar balik arah gerak agar tidak mudah diprediksi musuh
```

```
    waktu_putar_balik ← waktu sekarang
```

```
  ENDIF
```

```
  Buat beberapa kandidat arah gerak:
```

```
    - JIKA mode agresif: buat kandidat arah yang mendekati target_terbaik
```

```
    - JIKA mode normal: buat kandidat arah menyamping (mengorbit kiri/kanan) dari target_terbaik
```

```
  gerak_terbaik ← kandidat pertama
```

```
  skor_terbaik ← -∞
```

UNTUK setiap kandidat gerak LAKUKAN

Prediksi posisi bot jika maju sejauh 12 unit ke arah kandidat tersebut

Hitung skor gerak berdasarkan:

- Penalti nilai yang besar jika posisi baru terlalu dekat dengan dinding arena
- Bonus nilai jika posisi baru berada pada jarak ideal (sekitar 250 unit) dari target_terbaik

JIKA skor_gerak > skor_terbaik MAKA

skor_terbaik ← skor_gerak

gerak_terbaik ← kandidat gerak

ENDIF

ENDFOR

Putar badan bot ke arah gerak_terbaik

Gerakkan bot dengan kecepatan gerak_terbaik dikalikan dengan arah_gerak

ENDIF

ENDWHILE

Pada implementasi GreedyBot, struktur data utama yang digunakan untuk menyimpan informasi lawan adalah sebuah *Class* kustom bernama Enemy dan struktur koleksi Dictionary. *Class* Enemy bertugas merekam atribut spesifik dari setiap musuh secara individual seperti koordinat spasial (X, Y), sisa energi, kecepatan, arah gerak, serta waktu pemindaian terakhir. Objek-objek musuh ini kemudian dikumpulkan ke dalam memori Dictionary yang menggunakan ID musuh sebagai kunci pemetaan (*key*). Struktur data ini dipertahankan karena memfasilitasi proses pencarian, pembaruan, dan penghapusan data musuh dengan kompleksitas waktu yang sangat cepat. Bot ini digerakkan oleh fungsi heuristik GreedyPickTarget() yang bertugas mengembalikan objek target paling optimal. Fungsi ini bekerja dengan menyeleksi seluruh musuh di memori dan menetapkan skor ancaman secara *greedy* murni berdasarkan kalkulasi bobot energi lokal tertinggi. Setelah target terkunci, bot menjalankan serangkaian prosedur operasional di dalam prosedur utama Tick(). Prosedur operasional tersebut meliputi Radar() untuk mengunci fokus pemindaian pada target, Shoot() untuk menembakkan peluru secara prediktif ke lokasi masa depan target dengan kekuatan yang menyesuaikan jarak, serta Move() yang mengevaluasi sekumpulan kandidat arah gerak demi mempertahankan jarak tembak ideal sekaligus menghindari dinding arena.

4.4 Pengujian

Pengujian dilakukan sebanyak 3 kali dengan format pertandingan 1 vs 1 vs 1 vs 1. Tujuan dari pengujian ini adalah untuk melihat performa masing-masing bot berdasarkan strategi greedy yang digunakan. Hasil pengujian akan ditampilkan dalam bentuk dokumentasi gambar dari setiap pertandingan dengan menunjukkan hasil skor pertaningan yang menjadi dasar untuk membandingkan efektifitas strategi antar Bot.

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	suruh ai berpikir 1.1	2134	550	30	840	97	560	56	5	2	0
2	suruh ai berpikir-2 1.2	1926	1000	120	606	39	132	29	1	6	2
3	suruh ai berpikir-3 1.0	1242	500	60	540	43	84	15	2	1	3
4	suruh ai berpikir-4 1.0	826	500	30	208	13	74	0	1	0	4

Gambar 4.1 Hasil pengujian pertandingan pertama

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	suruh ai berpikir 1.1	3551	1000	90	1287	204	844	126	8	2	0
2	suruh ai berpikir-2 1.2	1505	800	120	373	56	156	0	0	4	3
3	suruh ai berpikir-3 1.0	1396	600	30	562	20	184	0	0	3	4
4	suruh ai berpikir-4 1.0	1328	600	60	470	26	152	19	2	1	3

Gambar 4.2 Hasil pengujian pertandingan kedua

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet Dmg.	Bullet Bonus	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	suruh ai berpikir 1.1	3669	950	120	1390	137	859	212	8	0	2
2	suruh ai berpikir-4 1.0	1508	800	60	556	8	83	0	0	5	3
3	suruh ai berpikir-3 1.0	1404	550	60	464	51	242	37	1	3	1
4	suruh ai berpikir-2 1.2	1272	650	30	390	27	174	0	1	2	4

Gambar 4.3 Hasil pengujian pertandingan ketiga

Berdasarkan Ketiga hasil pengujian diatas, performa setiap bot dapat dibandingkan dari konsistensi skor, kemampuan bertahan hidup, serta efektifitas strategi greedy yg diterapkan, Bot dengan hasil yang paling stabil dan skor tertinggi pada beberapa pengujian dapat dianggap memiliki strategi yang efektif dibandingkan bot lainnya.

BAB 5 : Kesimpulan dan saran.

5.1 Kesimpulan :

Berdasarkan hasil yang telah di implementasi dan pengujian bot pada robotcode itu sendiri, bisa disimpulkan bahwa algoritma greedy dapat diterapkan secara efektif dalam pengambilan keputusan secara real time. Setiap bot dengan pendekatan greedy yang heuristik berbeda untuk memilih target, menentukan daya tembak dan serta mengatur pergerakan.

Dari keempat bot yang sudah dibuat kita bisa menyimpulkan bahwa performa yang terbaik dengan skor tertinggi yaitu adalah bot pertama kita yaitu rammfire jr, dengan hal ini dapat menunjukkan bahwa pendekatan yang sederhana yang berfokus pada target serta serangan yang agresif dapat memberikan hasil yang optimal dalam skenario pengujian.

Sementara itu, alternatif lain memberikan keunggulan dari sisi efisien dan juga keseimbangan strategi namun cenderung kurang agresif dapat menghasilkan eliminasi yang cepat. Dengan demikian kesimpulan dalam permainan Robocode Tank Royale, pemilihan heuristik greedy yang tepat sangat berpengaruh pada hasil akhir dan performa bot.

5.2 Saran :

Untuk saran mungkin bisanya meningkatkan performa bot. Kombinasikan strategi greedy (hybrid) dapat dieksplorasi misalnya menggabungkan agresivitas dari alternatif 1 dengan efisiensi energi dari alternatif 3 yang menghasilkan bot yang jauh lebih seimbang. Penyesuaian parameter dapat bisa dilakukan secara lebih sistematis contohnya melalui eksperimen yang lebih banyak, pengujian bisa dapat diperluas dengan lebih banyak skenario dan lawan untuk memastikan bahwa strategi yang sudah digunakan tidak hanya optimal pada kondisi tertentu tetapi juga konsisten dalam berbagai macam situasi.

Lampiran

Link Repo Github :

https://github.com/Damble0/Tubes_Suruh_ai_berpikir

Youtube link :

<https://youtu.be/uhQaUEcglSQ>

Daftar Pustaka

Robocode. (n.d.). Robocode Tank Royale. Diakses pada 20 Mei 2026,
<https://robocode.dev/>